

System Architecture & Workflows

Complete Specification & Visual Diagrams Manual

Document Title:	End-to-End Application Workflow Specifications
Total Workflows Covered:	12 Core System Workflows
Security Standards:	OAuth2 / Signed JWT / Bcrypt Hashing / RBAC / Audit Trail
Document Version:	v1.0 (Production Architecture Ready)

Executive Summary

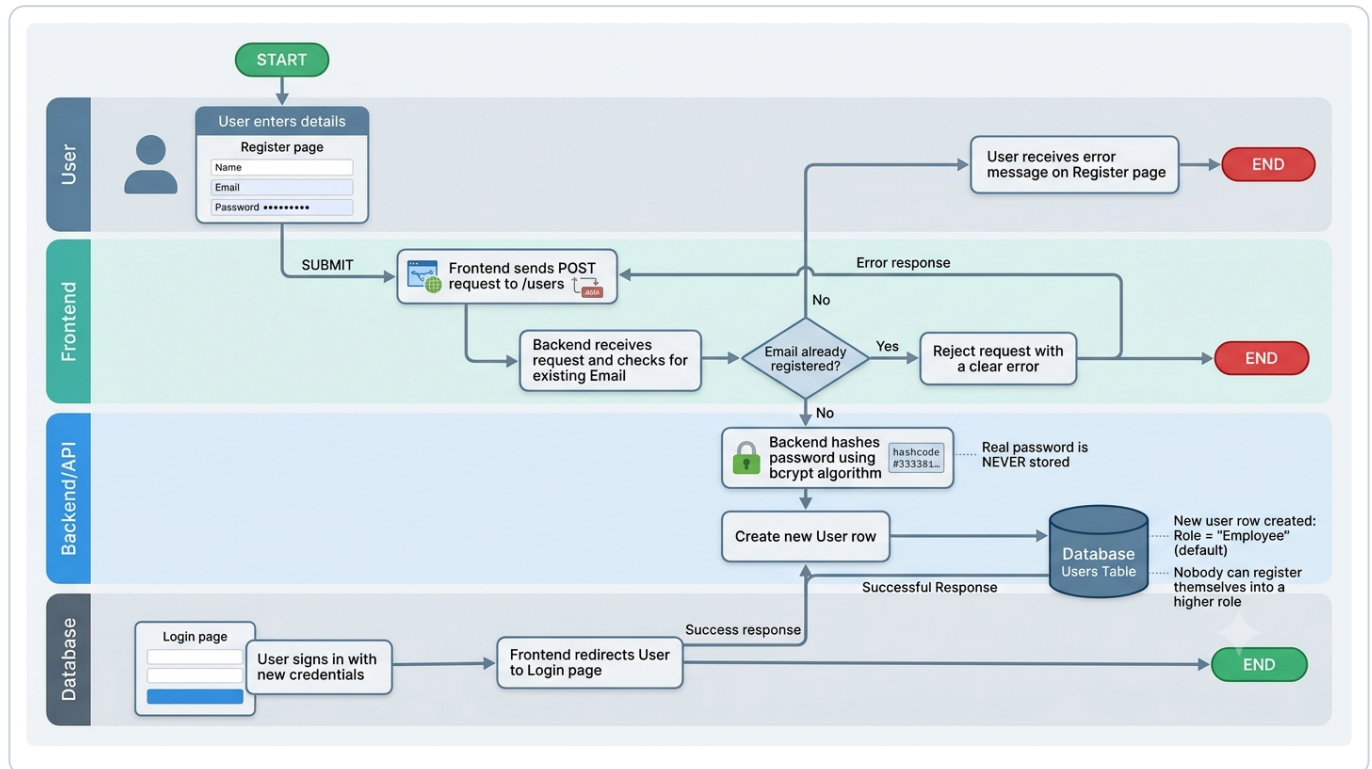
This technical manual provides a comprehensive specification and visual architecture diagram suite for all 12 system workflows. It covers authentication, authorization, role-based access control, organizational management, decision lifecycles, sequential multi-level approvals, audit tracking, attachment management, and dynamic data exports.

Registration Workflow

Authentication & Onboarding

Endpoint: POST /users

User onboarding process handling user registration, duplicate email checking, secure password hashing using bcrypt, role assignment, and redirection.



Process Execution Steps

1. User submits name, email, and password through the Register page.
2. Frontend sends payload as a POST request to /users.
3. Backend checks whether the email is already registered; if so, the request is rejected with a clear error response.
4. If the email is new, the password is hashed using bcrypt (a one-way hashing algorithm) so raw credentials are never stored.
5. A new User row is created with the role defaulted to 'Employee' — preventing unauthorized privilege escalation.
6. User is redirected to the Login page to sign in with their new credentials.

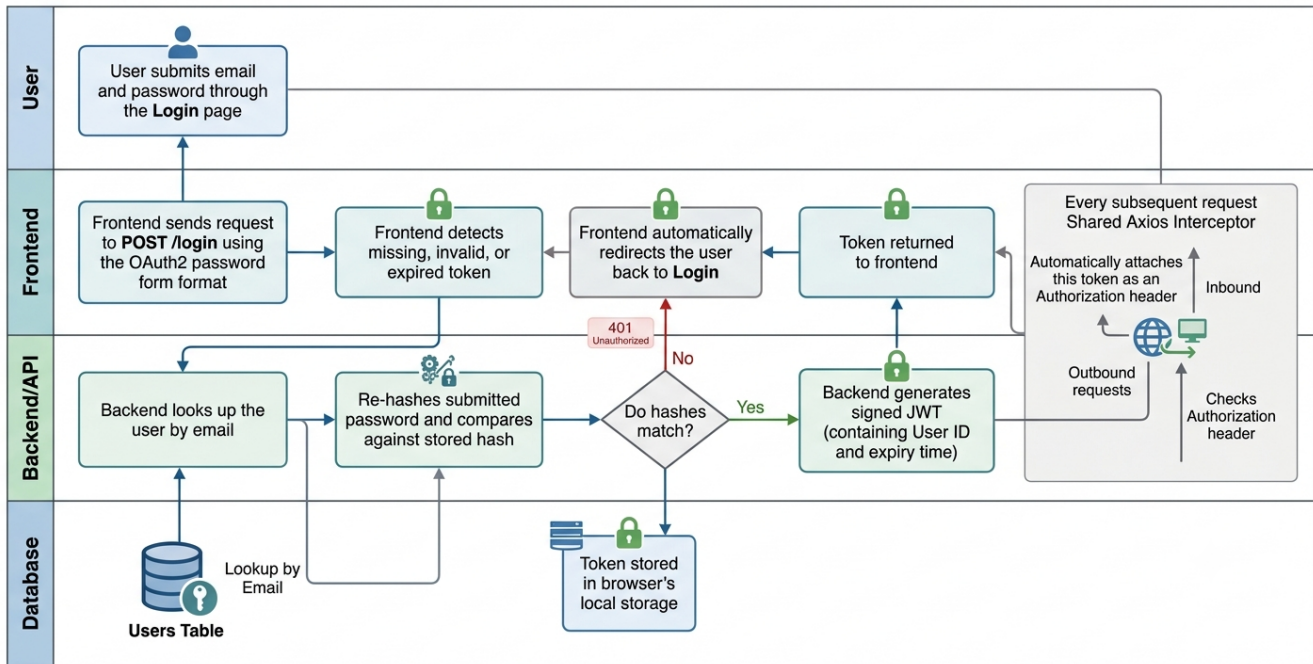
Login Workflow

Authentication & Session

Endpoint: POST /login

User session initiation using OAuth2 password form format, credential verification, and JWT generation stored in local storage.

2. Login Workflow



Process Execution Steps

1. User submits email and password through the Login page.
2. Frontend sends payload to POST /login using OAuth2 password form format.
3. Backend looks up user by email, re-hashes submitted password, and compares against stored bcrypt hash.
4. If credentials match, backend issues a signed JWT (JSON Web Token) containing user ID, role, and expiration timestamp.
5. Token is returned to frontend and stored in browser local storage.
6. Every subsequent request automatically attaches this token as an Authorization header via a shared Axios interceptor.
7. If token is missing, invalid, or expired, backend responds with 401 Unauthorized, triggering an automatic frontend redirect to Login.

Role-Based Access Control (RBAC)

Authorization & Security

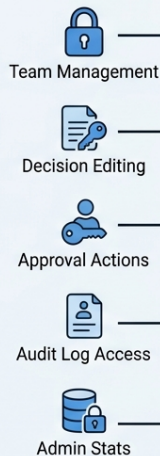
Endpoint: Middleware / Dependencies

System-wide authorization mechanism enforcing four distinct roles: Employee, Reviewer, Manager, and Administrator across all protected endpoints.

3. Role-Based Access Control (RBAC)

API Request Handling

Protected Endpoints

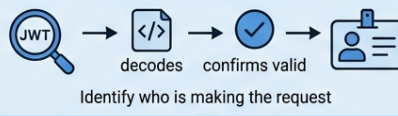


Incoming Request
(includes JWT)

RBAC Authorization Logic

Every Request Must Pass These Two Reusable Backend Dependencies

1. get_current_user



2. require_admin / explicit role checks



403 Forbidden Reject

Reject request with a clear message: user's role does not have permission

Action Allowed

Process Execution Steps

1. Every user is assigned exactly one role: Employee, Reviewer, Manager, or Administrator.
2. `get_current_user`: Decodes the incoming JWT, validates signature and expiration, and identifies request sender.
3. `require_admin / explicit role checks`: Evaluates user's role against permissions required for specific action.
4. If user role lacks necessary permission, request is rejected with 403 Forbidden.
5. Unified mechanism reused across Team Management, Decision Editing, Approvals, Audit Logs, and Admin Statistics.

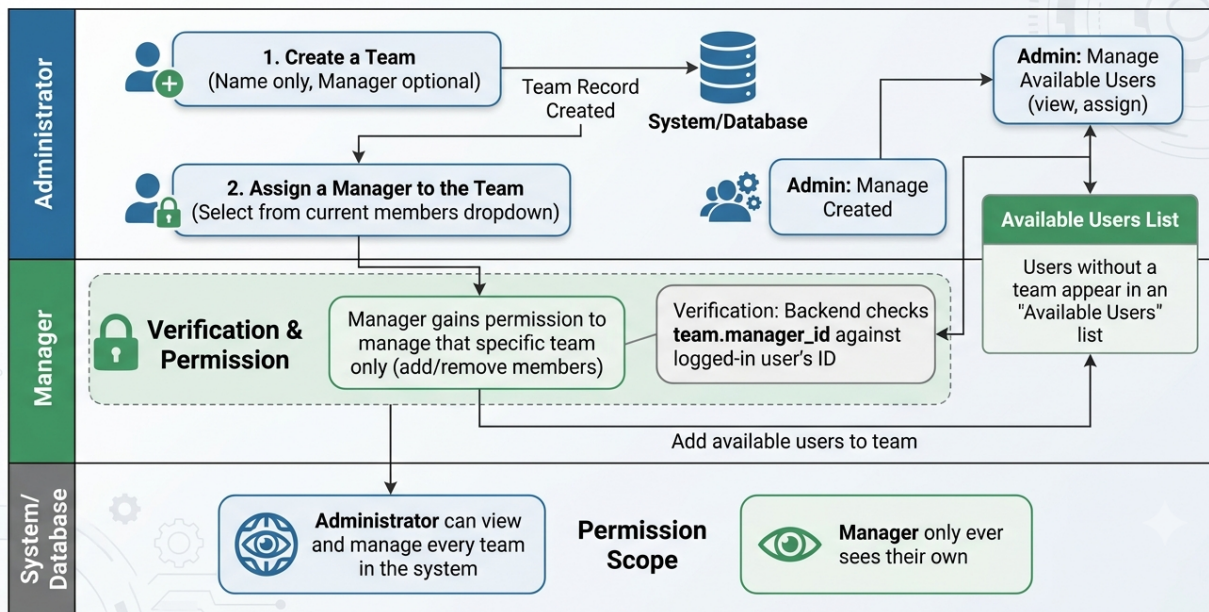
Team Management Workflow

Organizational Structure

Endpoint: POST /teams | PUT /teams/{id}

Hierarchical team structure setup, manager assignment, and localized administrative boundaries.

4. Team Management Workflow



Process Execution Steps

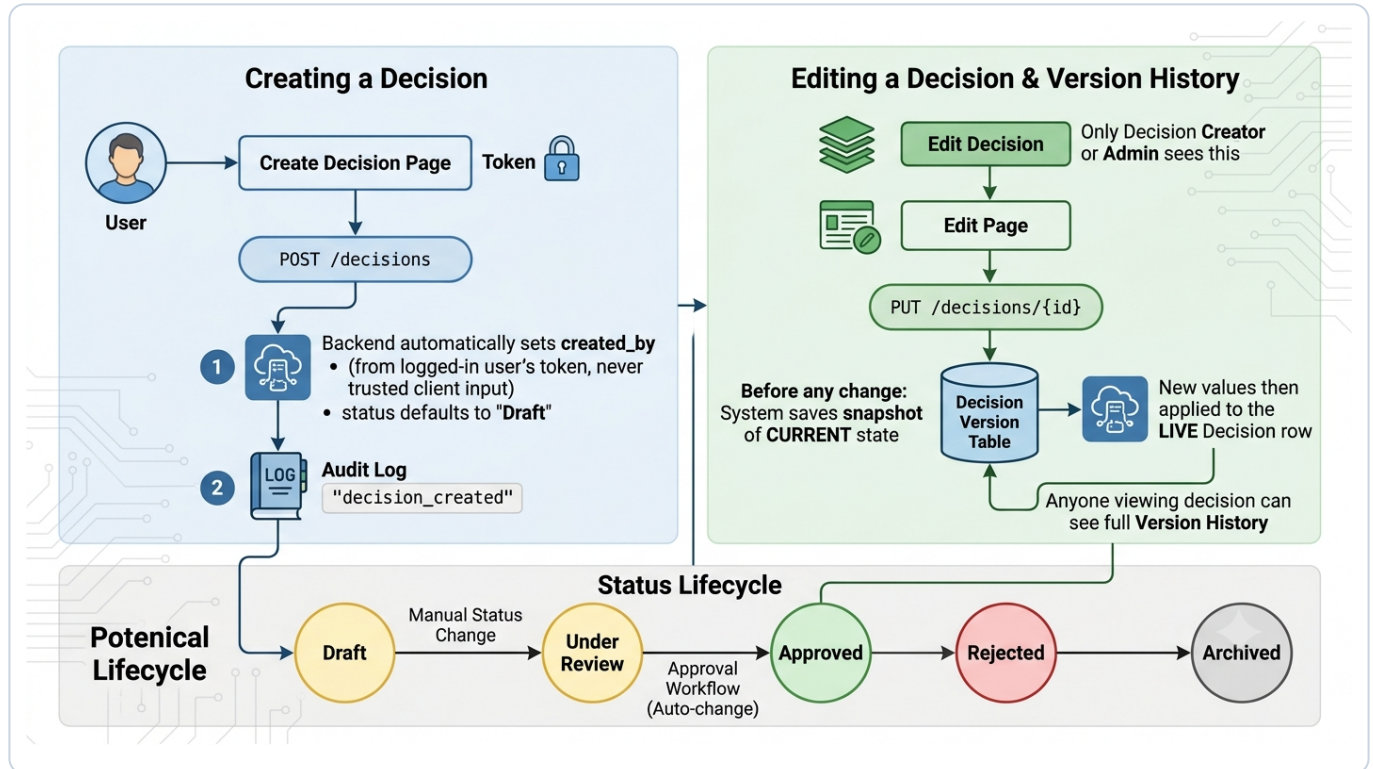
1. Administrator creates a Team (name required, manager optional initially).
2. Administrator assigns a Manager to team by selecting from current team members via dropdown.
3. Assigned Manager gains permissions to manage that specific team only (add/remove members), verified by `team.manager_id == user.id`.
4. Unassigned users appear in 'Available Users' list, accessible to Managers and Administrators for team addition.
5. Administrators maintain system-wide visibility, while Managers are scoped strictly to their assigned team.

Decision Management Workflow

Core Operations

Endpoint: **POST /decisions** | **PUT /decisions/{id}**

Creation, revision tracking, status transitions, and automated version history snapshotting for system decisions.



Process Execution Steps

1. User fills in Title and Problem Statement on Create Decision page; request sent to POST /decisions.
2. Backend sets created_by automatically from user's JWT token (client payload untrusted) and defaults status to 'Draft'.
3. Action recorded in Audit Log as 'decision_created'.
4. Editing allowed only for decision creator or Administrator.
5. Before applying edits, system captures snapshot of current state into DecisionVersion table (title, problem statement, status, author).
6. New values updated on live Decision row; full Version History available sequentially to all viewers.
7. Status Lifecycle follows: Draft → Under Review → Approved / Rejected → Archived.

Alternative Analysis Workflow

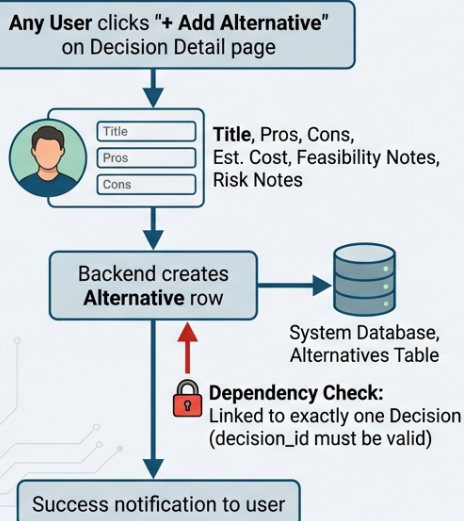
Decision Support

Endpoint: `POST /decisions/{id}/alternatives`

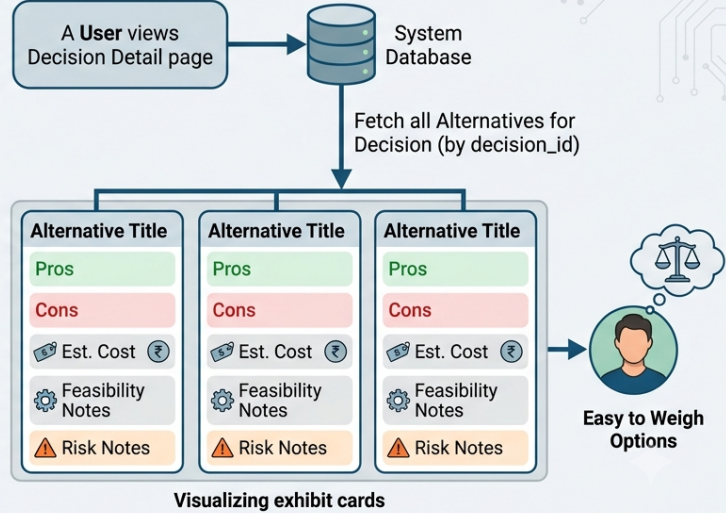
Comparative option modeling attached to specific decisions for multi-criteria evaluation.

6. ALTERNATIVE ANALYSIS WORKFLOW

Workflow to Add Alternative



Workflow to Display Alternatives



Process Execution Steps

1. Users add Alternatives from Decision Detail page including Pros, Cons, Estimated Cost, Feasibility Notes, and Risk Notes.
2. Each Alternative links to exactly one Decision via decision_id foreign key constraints.
3. Alternatives cannot exist independently in the system.
4. All Alternatives display side-by-side as comparative exhibit cards for structured decision-making.

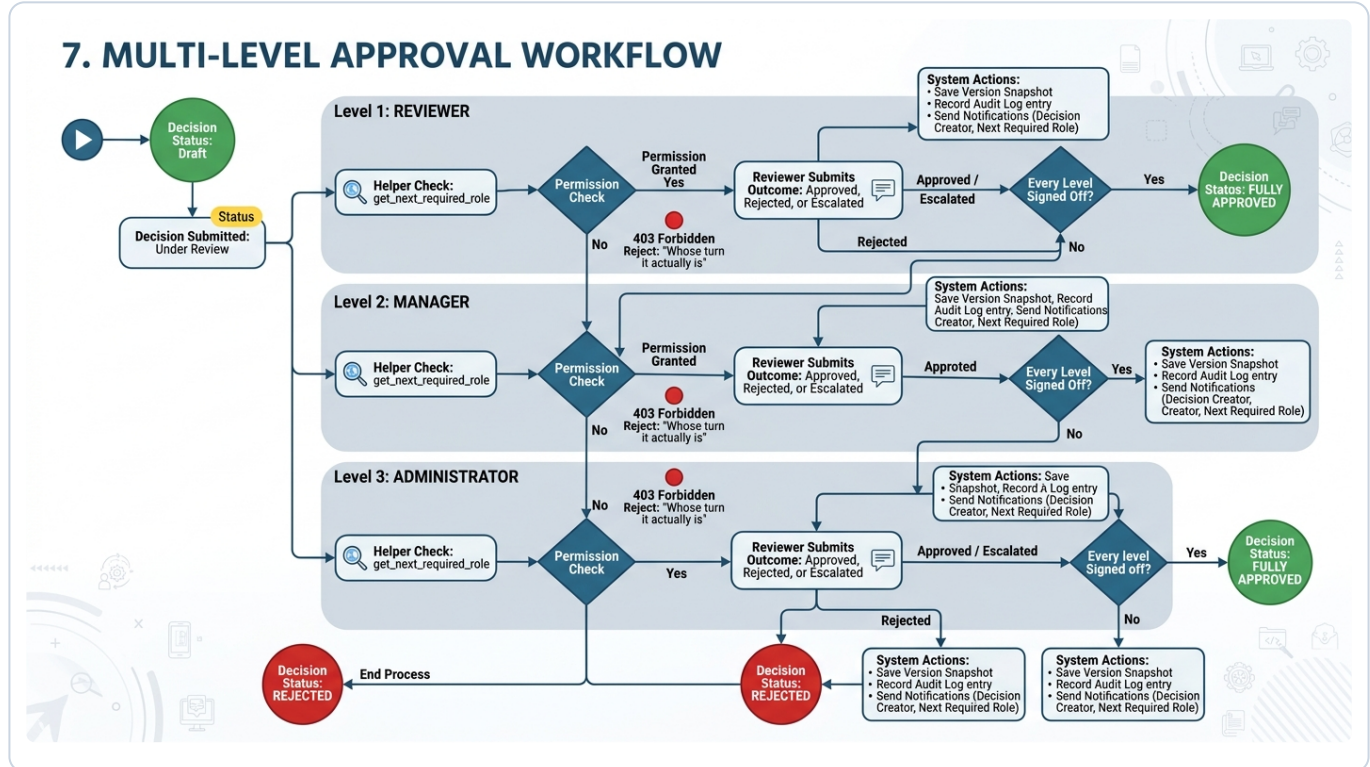
Multi-Level Approval Workflow

Governance & Compliance

Endpoint: POST /decisions/{id}/approvals

Three-stage sequential review pipeline (Reviewer → Manager → Administrator) with strict role gatekeeping.

7. MULTI-LEVEL APPROVAL WORKFLOW



Process Execution Steps

1. Decision must pass through three sequential levels: Reviewer, Manager, and Administrator.
2. System determines next required role via `get_next_required_role` helper.
3. If user role does not match required level, action blocked with 403 Forbidden indicating whose turn it is.
4. Reviewer submits outcome: Approved, Rejected, or Escalated with optional comment.
5. If Rejected, decision status becomes 'Rejected' immediately; workflow terminates.
6. If Approved/Escalated, system checks if all levels completed. If yes → 'Approved'; if remaining → 'Under Review' and advances level.
7. Saves Version snapshot, logs Audit entry, and sends dual Notifications to creator and next reviewer.

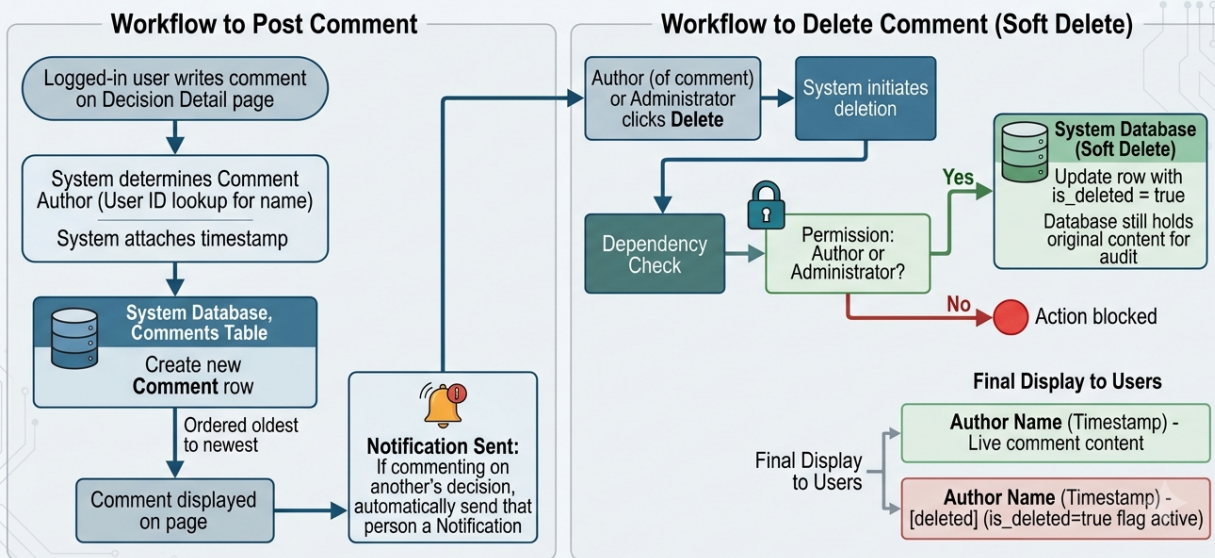
Discussion & Comments Workflow

Collaboration

Endpoint: POST /comments | DELETE /comments/{id}

Threaded user feedback on decisions featuring soft-deletion and automated author notifications.

8. DISCUSSION & COMMENTS WORKFLOW



Process Execution Steps

1. Logged-in users post comments on Decision Detail pages.
2. Comments display author name (looked up via user ID) and timestamp, sorted chronologically.
3. Comment author or Administrator can delete comments via SOFT delete (`is_deleted = true`).
4. Deleted comments render as '[deleted]', maintaining original text in database for audit compliance.
5. Commenting on another user's decision triggers an automated Notification to that decision's creator.

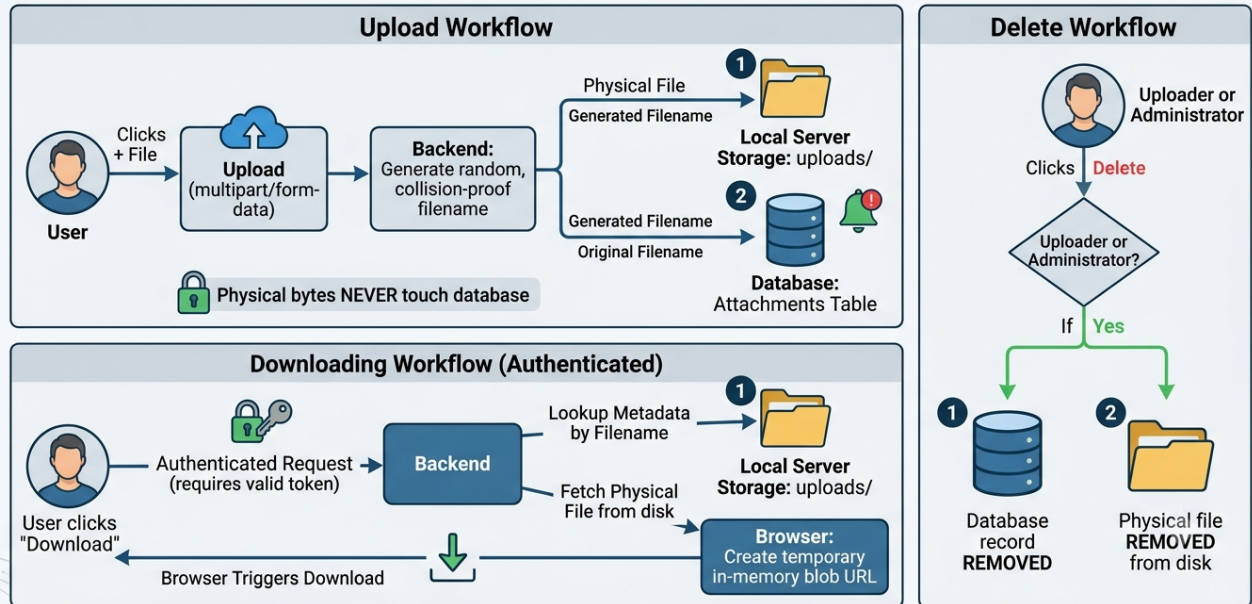
Attachments Workflow

Document Management

Endpoint: POST /attachments | GET /attachments/{id}

Multipart file upload handling, secure local storage, collision-proof filenames, and authenticated blob streaming.

9. ATTACHMENTS WORKFLOW



Process Execution Steps

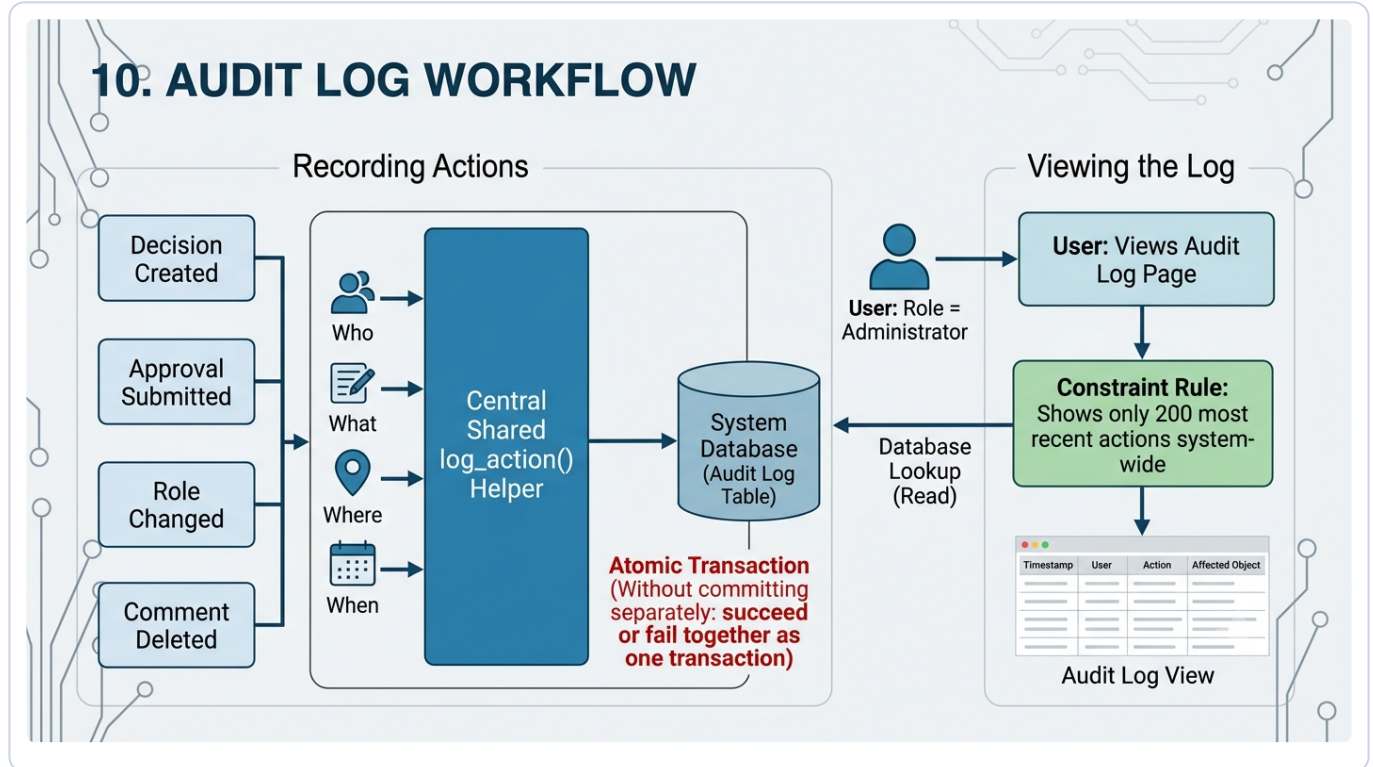
1. Files uploaded via multipart/form-data on Decision Detail page.
2. Backend generates collision-proof random filename and saves file to local uploads/ directory.
3. Database stores only metadata (generated filename, original filename); binary file bytes remain on filesystem.
4. Downloads require authenticated requests (JWT token validated); file served via temporary in-memory blob URL.
5. Uploader or Administrator can delete attachments, purging both database record and disk file.

Audit Log Workflow

Security & Traceability

Endpoint: GET /audit-logs

Centralized, atomic transactional logging for critical system actions accessible exclusively to Administrators.



Process Execution Steps

1. Central shared log_action() helper invoked during key actions (creations, approvals, role changes, comment deletions).
2. Logs Who, What, Where, and When in same transaction as target action to guarantee consistency.
3. Restricted to Administrators; displays top 200 recent system-wide actions.

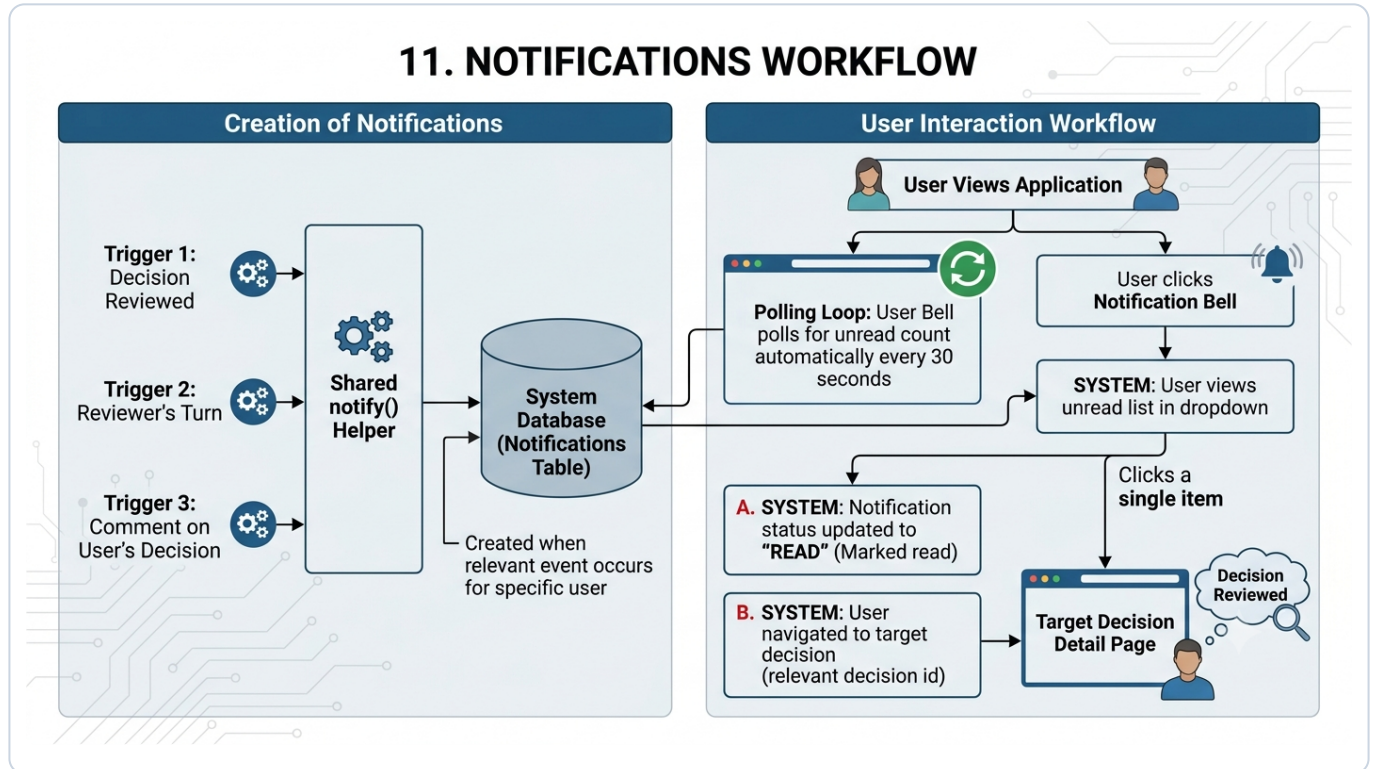
Notifications Workflow

User Engagement

Endpoint: GET /notifications | PATCH /notifications/{id}

Event-driven user alert creation, automatic UI polling, and direct navigation dispatch.

11. NOTIFICATIONS WORKFLOW



Process Execution Steps

1. Shared notify() helper generates Notification rows on key events (decision review, assigned turn, incoming comments).
2. Frontend notification bell automatically polls backend every 30 seconds for unread count.
3. Opening dropdown displays recent alerts; clicking item marks notification read and navigates directly to target decision.

Search, Filtering & Export Workflow

Data Retrieval & Reporting

Endpoint: GET /decisions | GET /decisions/export/pdf | GET /decisions/export/excel

Debounced live search query processing, multi-parameter filtering, and dynamic server-side document streaming.

 **Search & Debounce**

Debounce: 400ms pause post-keystroke.

Endpoint: GET /decisions?search=...

Backend: Case-insensitive partial match on title.

 **PDF Streaming Export**

Engine: ReportLab in-memory.

Endpoint: GET /decisions/{id}/export/pdf

Security: Direct browser stream; 0 server disk footprint.

 **Excel Data Export**

Engine: OpenPyXL spreadsheet writer.

Endpoint: GET /decisions/export/excel

Feature: Streams full decision table directly.

Process Execution Steps

1. Decision List page provides search box and status filter passed as optional query parameters to GET /decisions.
2. Search input is debounced (400ms pause) to prevent redundant network requests while typing.
3. Backend applies case-insensitive partial match on title and exact match on status, combining filters when active.
4. PDF report generated on-demand using ReportLab in memory and streamed directly as browser download without disk storage.
5. Excel spreadsheet generated using OpenPyXL and streamed dynamically from Decision List view.